A hitchhiker's guide to using & securing AI

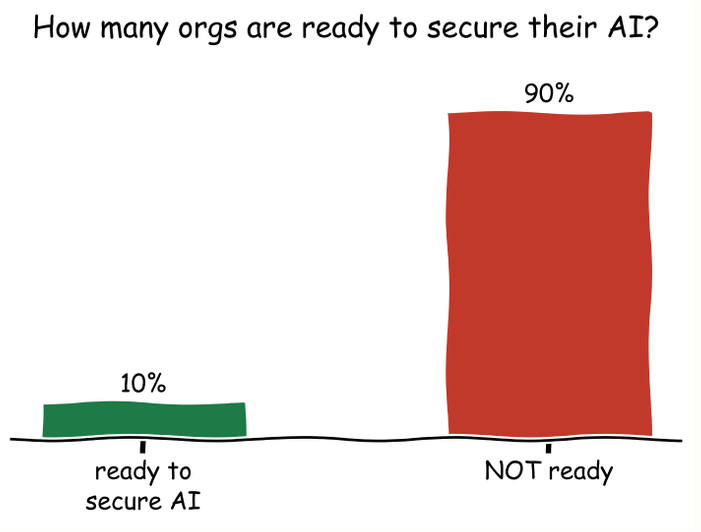
Don't Panic…and don't just vibe either

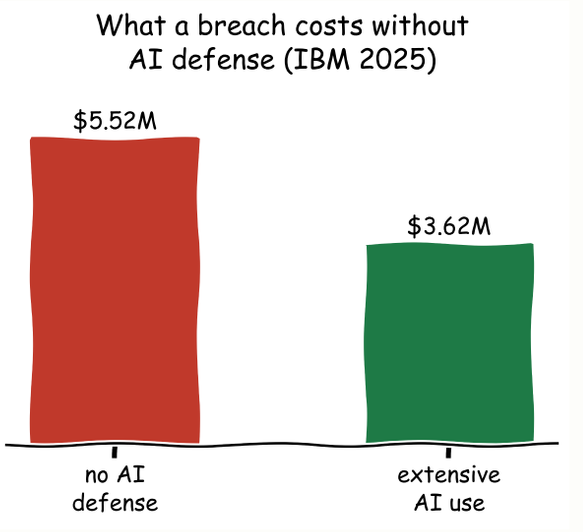
Audience: engineers & fellow travellers

A Recap of OWASP Appsec conference, Italy, June 17–18th, 2026

Bahar Partov· Security Engineering, Tucows· June 22nd, 2026 ·

Adoption is outrunning security





10% of orgs are ready to defend their AI — **90% are not**. Agents now act, not just talk — and the gap costs **$1.9M** more per breach.

Sources: Accenture, *State of Cybersecurity Resilience 2025* — 63% sit in the "exposed zone"; 77% lack core AI security practices. · IBM, *Cost of a Data Breach 2025* — global avg breach $4.44M.

THE DECK · 3 SECTIONS

SECTION 1 · Secure Coding with AI

how do you write code with AI?

- Vibe coding
- Spec-driven development

SECTION 2 · Securing agents

how do you secure what the agents do?

- Agentic & multi-agent flows
- Governance
- Guardrails + the NIST limit
- Prompt injection

SECTION 3 · AppSec at scale

how do you ship it securely, at scale?

- Trustworthy CI/CD pipeline
- AppSec program at scale

SECTION 1

Secure Coding with AI

Comic: after xkcd #303 "Compiling" (Randall Munroe, CC BY-NC 2.5)

§1.1

Vibe coding

Generating code is easy. Finding the bugs in it is not.

Betta Lyon Delsordo — AWS

What do you think vibe coding really *is*?

AI re-introduces classic bugs — at speed

🔑 **MD5 passwords**
`md5(pw)`

💉 **SQL injection**
`f"…WHERE u='{u}'"`

🗝️ **hard-coded secrets**
`KEY="sk-live-…"`

🎭 **fake MFA**
`verify()→True`

⚡ **reflected XSS**
`f"<b>{q}</b>"`

🚪 **auth fails open**
`if not row: ok`

📦 **typosquatted imports**
`import reqeusts`

🚀 **prototype → prod**
`DEBUG = True`

🔗 Real-world: **Lovable CVE-2025-48757** — AI generated Supabase configs with row-level security **off by default**; ~170 apps (1 in 10 of the marketplace) leaked user data & API keys, visible in browser devtools. writeup

"If you're not reading the code, you're vibe coding."

Spot the vuln an AI-written login

```
@app.post("/login")
def login(user, pw):
    row = db.execute(
        f"SELECT * FROM users WHERE name='{user}' AND pw=md5('{pw}')")
    if not row:
        return {"ok": True}            # TODO: real auth — fails OPEN
    return {"ok": True, "token": "debug-123"}
```

SQL injection — user text built into the query

MD5 — broken password hashing

fails open — no row → returns ok

hard-coded token

Four classic bugs in eight lines — none exotic. *This* is what "review the AI's code" actually means.

Spot the vuln

a vibe-coded MMO · ~45 commits · 5k-line files

```
// character state is persisted as one freeform JSONB blob…
await db.save(id, character)          // { pos, gear, …, is_gm }

// …so a field the player can set round-trips back as admin
if (character.is_gm) grantGodMode()   // privilege escalation

// and every WS handler reads straight toward the happy path:
const msg = JSON.parse(raw)           // frame = the literal `null`
switch (msg.t) { … }                  // null.t → crashes the server
```

privilege escalation — player-controlled admin flag

one-line DoS — a `null` frame kills every session

emergent — not on the 2005 OWASP list

The auth is actually solid — scrypt, `timingSafeEqual`, tokens in the body. AI nails the *textbook* answers; the **threat-model** bugs (JSONB-everything, happy-path handlers) slip through. levy-street/world-of-claudecraft

Make the secure way the default

SOUNDS OBVIOUS

Review AI-generated code.

THE CATCH

The bugs aren't exotic — it's the 2005 OWASP list, and the unfinished auth **fails open** (returns true). Reading ≠ understanding. The point: make each layer a **default**, not a good intention — so the bugs get caught even when no one's looking.

🔗 Betta's hands-on workshop (vulnerable app + slides): github.com/Betta-Lyon-Delsordo/insecure-vibes

§1.2

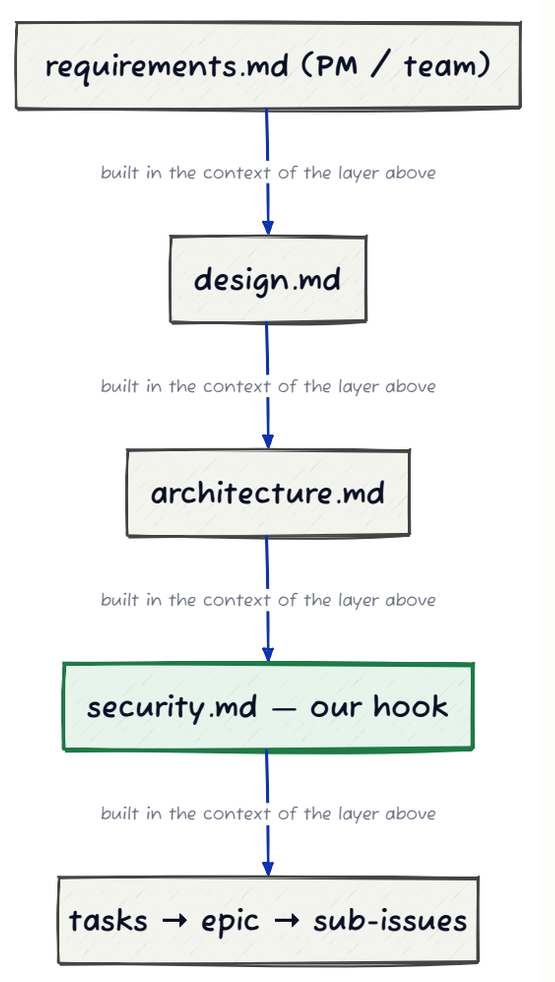
Spec-driven development

The disciplined counterpart to vibe coding

Jim Manico — Manicode Security

What's the disciplined opposite of vibe coding?

Build to a spec the AI can't drift from



Now an industry pattern — Spec Kit, Kiro, OpenSpec — born in 2025 as the answer to vibe coding.

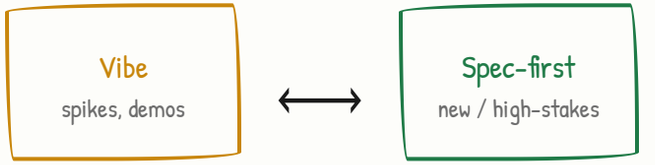
You write the spec; the AI builds to it

```
# requirements.md   (you write this)
- Auth: email + password
- Passwords: argon2id, min 12 chars
- Login: rate-limit 5 / min / IP
- Lock account after 10 failures
```

```
# security.md   (reviewed, ships with build)
- No secrets in code -> use the vault
- All SQL parameterized (never built by string)
- Log auth events; never log passwords
```

Change a rule, re-run — the AI builds to the **contract**, not a vibe. (Contrast the §1.1 login that did the opposite of every line here.)

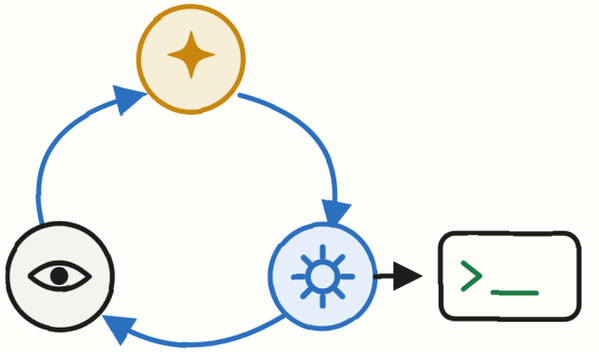
§1.2 · SPEC-DRIVEN DEV



Don't mandate the full ceremony. Borrow the spine: a short `requirements.md` + reviewed `security.md` before the AI writes code.

SECTION 2

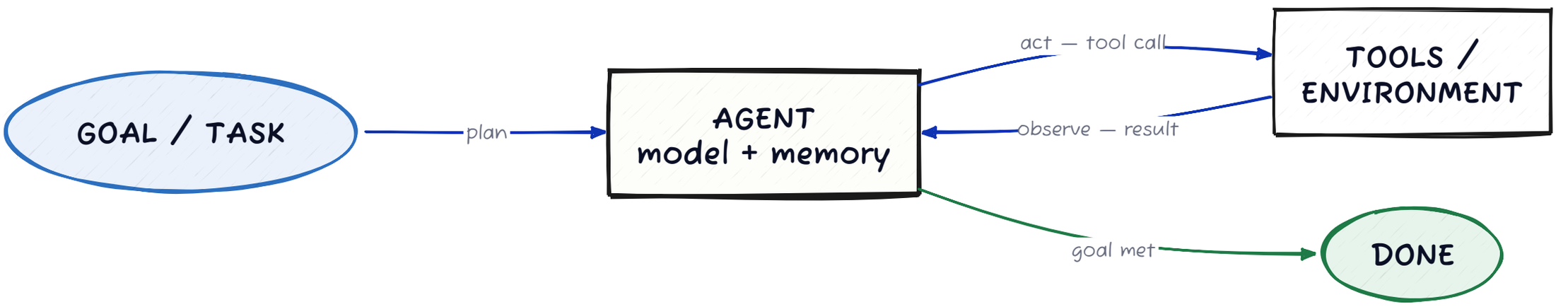
Securing agents



Agentic flows | Governance | Guardrails + NIST proof | Prompt injection

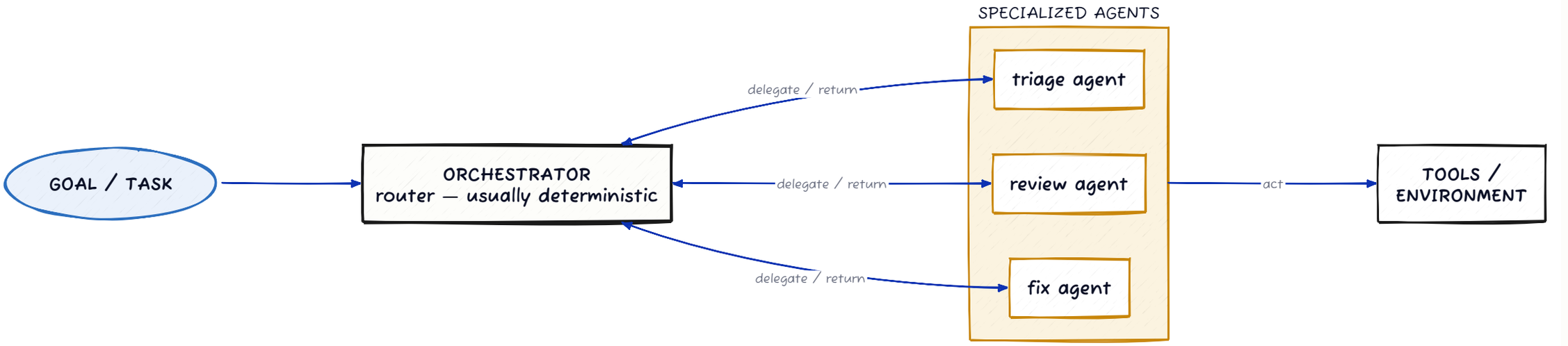
What *is* an agent — and what's **not** one?

An agentic flow is a loop



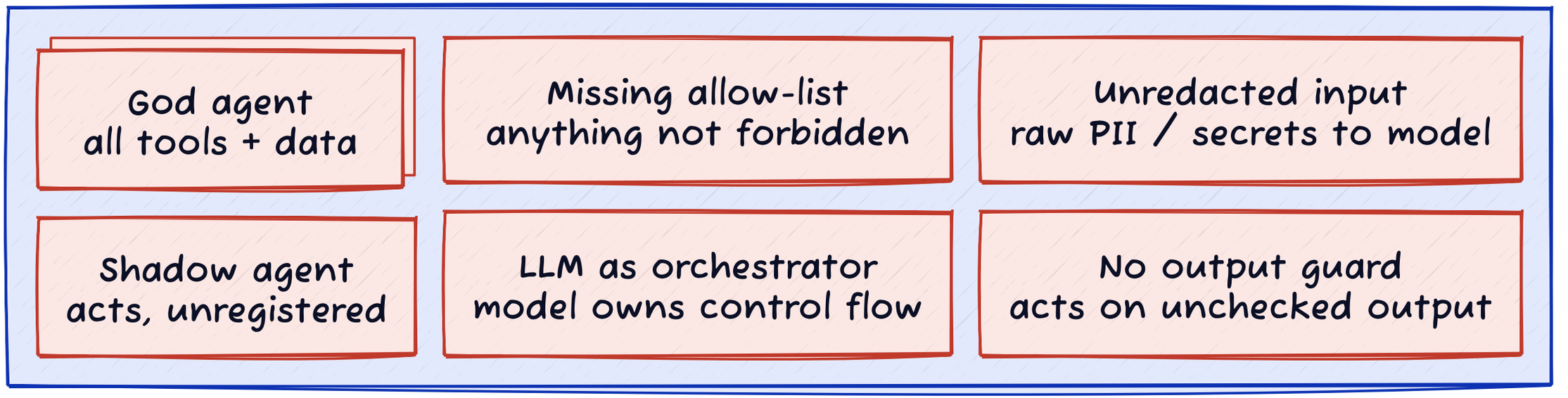
An **agent** = a model that **plans** → **acts (tool call)** → **observes** → **repeats** until the goal is met. A **skill / tool** is *not* an agent — it's a passive capability the agent **calls** (does one thing, no goal, no loop), and a trusted input it can be tricked through (→ §2.3). The power & the risk live in the agent's act step: it touches real tools & data.

…and a multi-agent flow is many loops, routed



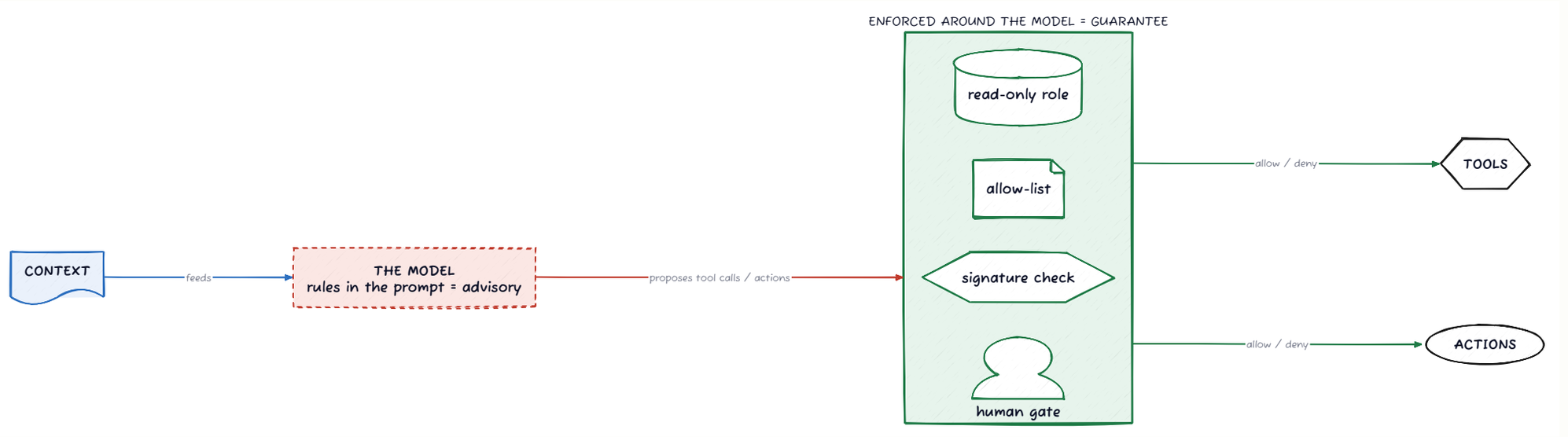
A **swarm** = an orchestrator routing a goal across **specialized agents**. More agents → more act steps, more privilege, more attack surface. *That's* what the rest of this section secures.

The antipatterns these flows fall into



From the OWASP GenAI / Agentic Security Initiative. Each one breaks a golden rule — **single responsibility · least privilege · deterministic-first · input mediation · output gating · auditability**. The rest of Section 2 is the fixes.

Around the model, not in it



The fix for every antipattern above: put enforcement **around** the model, not in its prompt 👇

§2.1

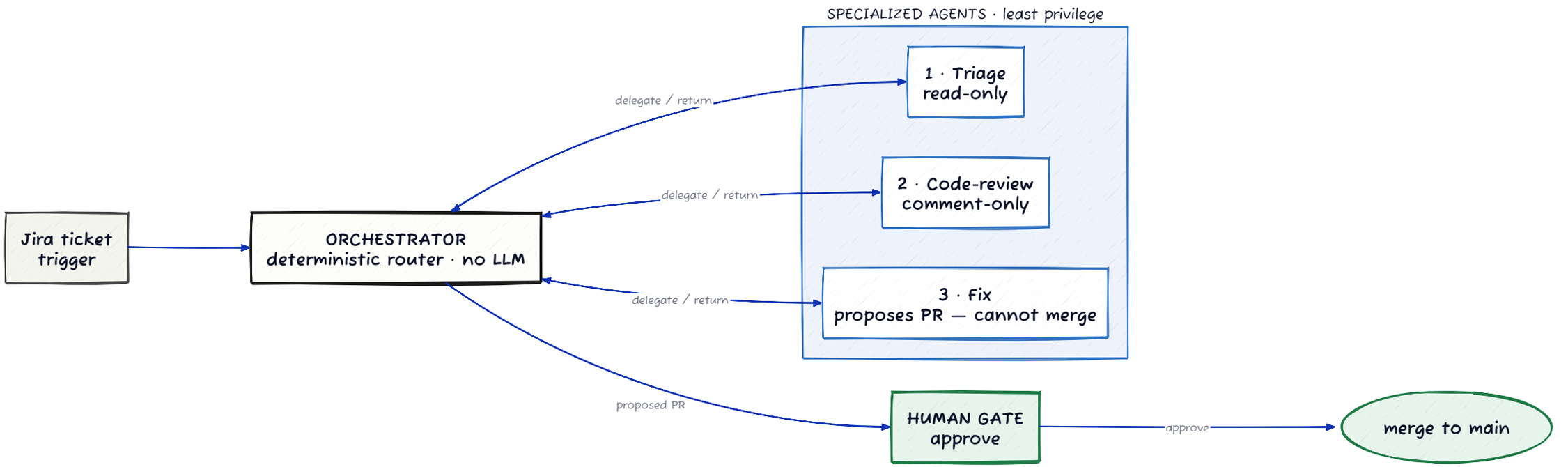
Agent governance

Govern agents like assets — register · identify · authorize · route

Secure AI Agent Swarm training

In a swarm, who should get the *most* privilege?

Multiagent code-review / Jira flow



💡 **The agents never talk to each other** — every call routes through the deterministic orchestrator. Privilege *shrinks* as capability grows: Fix is the smartest and the only one that touches the PR, yet it can't merge.

SOUNDS OBVIOUS

Give each agent least privilege.

THE CATCH

The **most capable** agent gets the **least** (Fix only proposes). A robust "swarm" is mostly deterministic code — **6 of 9** lab "agents" ran no model at all. The orchestrator routes; LLMs never coordinate.

Register every agent — no shadow agents

One entry per agent in a central registry — **identity, routing tier, and an explicit allow-list**. Everything not listed is denied by default.

```
- id: code-review-agent        # identity  → authenticate
  safety_level: high            # routing tier
  allowed_actions:              # authorization (allow-list)
    - read_repo
    - post_review_comment
  denied_actions:               # default-deny everything else
    - write_files               #   ✗
    - merge_pr                  #   ✗
```

registry.yaml
the policy — versioned, reviewed

→

Orchestrator routes by `id` + `safety_level`
no entry → **won't route** (kills shadow agents)

Action gate enforces `allowed_actions`
off-list call → `deny()` before it fires

Registry = **policy**; the deterministic **orchestrator + action gate enforce it**, around the model — the LLM never reads or grants its own permissions. Maps to OWASP Agentic Security Initiative + LLM06 Excessive Agency.

Specialized vs. general models: fit beats "smart"

Specialized / fine-tuned (small, often open-weight)
✓ wins on **narrow, well-defined tasks** — triage, classification
✓ Equixly's fine-tuned open-weight model **beat general models** on API/web triage — cheaper, faster, in-boundary
✓ one model per agent role → compartmentalize

General / frontier
✓ broader open-ended reasoning
✗ cost · knowledge cutoffs · data egress
✓ route the genuinely hard, fuzzy steps here

SOUNDS OBVIOUS

Use a security-specialized model for security.

THE CATCH

Specialization improves **fit**, not **trust**. A security-specialist 8B model still certified a critical heap-overflow as a "minor log leak." Right-size per task — and for *detection*, deterministic tools beat both.

§2.2

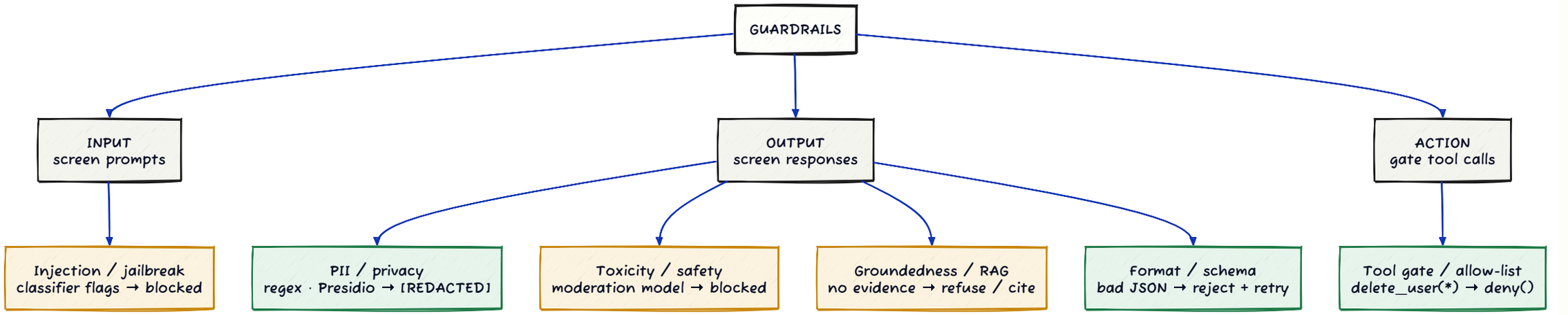
Guardrails

The enforcement layer around the model — and its hard limit

Secure AI Agent Swarm training · NIST 2026

How many *types* of guardrail are there?

One tree: by stage, by function, by enforcement

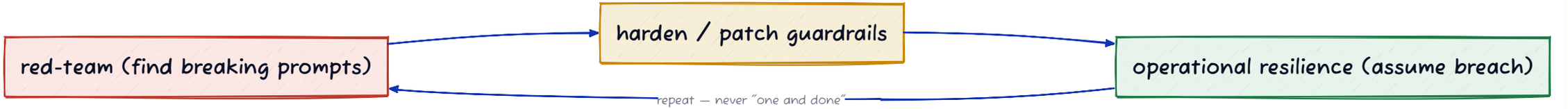


green = deterministic — guarantees what it covers · **amber = model-based** — broader, but foolable & itself injectable. Real systems **stack several.** Tooling: Llama Guard · Granite Guardian · NeMo · Lakera · Bedrock · Presidio.

Why guardrails are never the ceiling

> **"No finite set of guardrails is universally robust against adaptive prompts."**
>
> — NIST (A. Vassilev), *IEEE Security & Privacy* 2026 · "Robust AI Security & Alignment: A Sisyphean Endeavor?"

A **Gödel-incompleteness-style** proof: for *any* fixed guardrail set, some prompt exists that defeats it. So security can't be "one and done" →



Crucial: this is a different axis from slide 4. It holds for **any** finite guardrail set — **even deterministic filters**. Not "prompt rules are weak" — it's that *coverage can never be complete*. So: deterministic enforcement for what you anticipated **+ continuous red-team** for what you didn't.

 INTERACTIVE · CLICK THE BUTTON

The guardrail game: every fixed filter set loses

These are real **around-the-model** filters & classifiers — *not* prompt text (that's already advisory, slide 4). Even deterministic ones can't cover every input 👇

🛡️ Guardrail layer filters + classifiers, around the model

- input filter: block "ignore previous instructions"

😈 Attacker's next prompt

click ▶ to watch the attacker try…

▶ attacker tries again

🔗 Play it live: Lakera Gandalf · Tensor Trust (attack *and* defend) · Prompt Airlines — beat one guardrail, then watch the next one fall too.

§2.3

Prompt injection

When the thing the model reads becomes the thing it obeys

Matteo Grollino — Relatech · #1 in OWASP LLM Top 10

Where does the dangerous input actually come from?

Indirect injection: a review field → RCE

SOUNDS OBVIOUS

Validate untrusted user input.

THE CATCH

The dangerous input isn't what's *typed* — it's what the agent already **trusts**: its RAG, skills, memory, another agent's output. = OWASP LLM01 → LLM05 (Improper Output Handling).

🔗 Real-world: **CamoLeak** (GitHub Copilot Chat, CVE-2025-59145) — a hidden payload in a **PR description** made Copilot exfiltrate private source, keys & secrets via a trusted channel — *this slide's diagram, in production*. Also **EchoLeak** (CVE-2025-32711, zero-click M365 Copilot exfil). CamoLeak · EchoLeak

SECTION 3

AppSec at scale

Trustworthy CI/CD pipeline | AppSec program at scale

§3.1

Trustworthy CI/CD

Sign what you ship · verify before it runs

Adobe — zero-trust supply chain · 100K+ builds/day

§3.1 · TRUSTWORTHY CI/CD

How do you trust what's actually running in prod?

Sign at build → verify at deploy

💡 **Why keyless?** short-lived certs **expire before they can be abused** — no keys to steal, no revocation to run. The constraint *is* the security.

Already on pull-based ArgoCD? The next step is cosign-signing in CI + an admission controller (Kyverno / Sigstore) that refuses unsigned images.

Concretely: the attack it stops

Without signing
An attacker pushes a tampered image to the registry under the same tag → the cluster pulls it → **it runs in prod.** Nobody notices.

With cosign + admission control
The tampered image has **no valid signature** → the admission controller **denies the pod** → it never runs.

Same lesson as the §1.1 login: what holds isn't a policy doc — it's a check enforced at deploy, outside anyone's good intentions.

§3.2

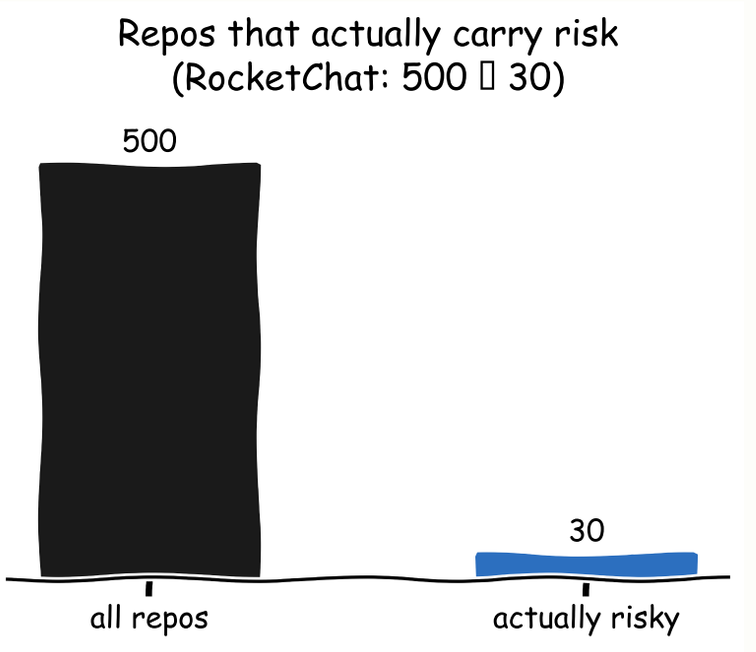
An AppSec program that scales

Built from scratch with 2 security engineers — sound familiar?

Julio Araujo — Rocket.Chat

2 engineers, 500 repos — where do you even start?

Focus the effort — don't chase findings



SOUNDS OBVIOUS

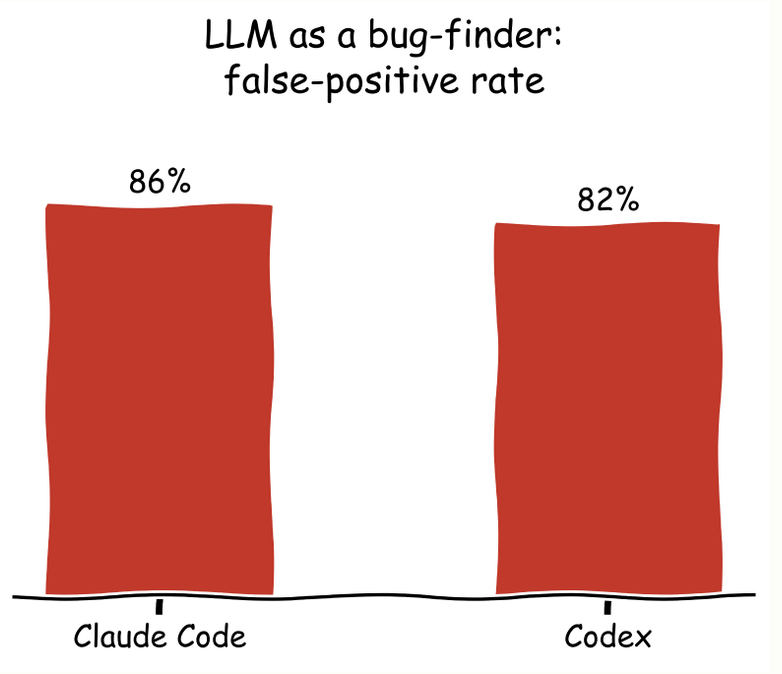
Educate developers, get buy-in.

THE CATCH

Security-as-blocker statistically **loses**. Leverage is structural: fix by **class** (one control kills ~90% of a type) + triage to the repos that actually carry risk. Paved roads, not roadblocks.

Concretely: 40+ SSRF findings across services → one shared hardened HTTP client everyone imports → **~90% closed in a single PR**, instead of 40 tickets.

Don't make an LLM the *sole* bug detector



It didn't *miss* — it certified a critical heap-overflow as a minor log leak, and it's non-reproducible (3 runs → 3 disjoint FP sets). Rules detect; the LLM verifies & explains. · Where local LLMs aren't an option → approved cloud + de-identify before egress.

TL;DR

Use AI deliberately

vibe for spikes · spec for what matters

Govern the agents

register · scope · sign · human gate

Enforce it around the model — never in it.

Don't panic. Don't just vibe. Bring a towel (and a guardrail). 👨‍🚀

SOURCES & FURTHER READING

✓ FACT-CHECKED JUN 2026

- **Vibe coding workshop** (Betta Lyon Delsordo) — github.com/Betta-Lyon-Delsordo/insecure-vibes
- **OWASP Top 10 for LLM Apps 2025** — LLM01 Injection · LLM05 Improper Output Handling · LLM06 Excessive Agency — genai.owasp.org
- **OWASP Agentic Security Initiative** (ASI) — genai.owasp.org/initiatives/agentic-security-initiative
- **Guardrail limits** — A. Vassilev (NIST), *IEEE S&P* 2026, "Robust AI Security & Alignment: A Sisyphean Endeavor?" — nist.gov · arXiv:2512.10100
- **Guardrail tooling** — Llama Guard · IBM Granite Guardian · NVIDIA NeMo Guardrails · Lakera · Bedrock Guardrails · Presidio
- **Supply chain** — Sigstore / cosign · Kyverno / policy-controller · **Semgrep MCP** · **Spec-driven** — Spec Kit · Kiro · OpenSpec
- **LLM bug-finding reliability** — ~80%+ false positives; hybrid static-analysis + LLM (ZeroFalse, arXiv 2510.02534)
- **AI security-readiness gap** — Accenture, *State of Cybersecurity Resilience 2025* (only 10% ready)
- **Spec-driven dev** — GitHub Spec Kit · methodology (spec-driven.md) · launch blog
- **Long-context cost** — "Lost in the Middle", Liu et al. 2023 (TACL) · "Less Context, Better Agents" 2026 (context-optimized agents: 2.68× fewer tokens, no quality loss)
- **Try it live** — Lakera Gandalf · Tensor Trust · Prompt Airlines · HackAPrompt · PortSwigger Web LLM labs
- **Real incidents** — CamoLeak / CVE-2025-59145 (Copilot Chat exfil) · EchoLeak / CVE-2025-32711 · ForcedLeak (Agentforce CRM exfil) · Chevy $1 car · Lovable CVE-2025-48757

Talks: Verma (Snyk) · Grollino (Relatech) · Lyon Delsordo (AWS) · Manico · Adobe · Araujo (Rocket.Chat)